

FAME: FPGA Acceleration of Secure Matrix Multiplication with Homomorphic Encryption

Zhihan Xu*, Rajgopal Kannan[†] and Viktor K. Prasanna*

*University of Southern California, USA

[†]DEVCOM Army Research Office

Email: {zhihanxu, prasanna}@usc.edu, rajgopal.kannan.civ@army.mil

Abstract—Homomorphic Encryption (HE) enables secure computation on encrypted data, addressing privacy concerns in cloud computing. However, the high computational cost of HE operations, particularly matrix multiplication (MM), remains a major barrier to its practical deployment. Accelerating Homomorphic Encrypted MM (HE MM) is crucial for applications such as privacy-preserving machine learning.

In this paper, we present a bandwidth-efficient FPGA implementation of HE MM. We first develop a cost model to evaluate the on-chip memory requirement for a given set of HE parameters and input matrix sizes. Our analysis shows that optimizing on-chip memory usage is critical for scalable and efficient HE MM. To this end, we design a novel datapath for Homomorphic Linear Transformation (HLT), the major bottleneck in HE MM. Our datapath significantly reduces off-chip memory traffic and on-chip memory demand by enabling fine-grained data reuse. Leveraging the proposed datapath, we introduce FAME, the first FPGA-based accelerator specifically tailored for HE MM. FAME supports arbitrary matrix shapes and is configurable for a wide range of HE parameter sets. We implement FAME on Alveo U280 and evaluate its performance over diverse matrix sizes and shapes. Experimental results show that FAME achieves an average of $221\times$ speedup over state-of-the-art CPU-based implementations, demonstrating its scalability and practicality for large-scale consecutive HE MM and real-world workloads.

Index Terms—FPGA Accelerator, Secure Matrix Multiplication, Homomorphic Encryption

I. INTRODUCTION

Cloud computing has revolutionized many applications across various sectors such as healthcare, finance, and government [1], [2]. Despite its computational advantages, security and data privacy remain concerns in adopting cloud computing services [3]. Homomorphic Encryption (HE) offers a promising solution by enabling computations directly on encrypted data without decrypting the data at the server side. However, HE introduces substantial computational overhead, often several orders of magnitude higher than equivalent plaintext operations [4]. As a result, accelerating HE applications has become a focal point of recent research efforts [5]–[12].

Existing HE acceleration efforts [5]–[12] primarily focus on scenarios where the machine learning (ML) model remains in plaintext and only the input data is encrypted. For example, works such as [10]–[12] accelerate matrix-vector multiplication where the matrix (i.e., the model) is unencrypted. While this setup reduces computational cost, it is not sufficient for applications requiring stronger privacy guarantees, where both

the model and data must remain encrypted. Two representative scenarios illustrate this need: (1) a cloud service provider owns the model, trains the model with the encrypted data from the data owner, and uses the encrypted trained model to make inferences on new encrypted inputs; (2) a third-party model provider (e.g., researchers or companies) owns the trained encrypted model and uploads it to the cloud service provider to perform inference on encrypted inputs from various data owners. In both scenarios, both the model and the input data are encrypted, necessitating secure and efficient HE matrix multiplication (MM) to support fully encrypted inference workflows.

HE MM is defined as matrix multiplication in which both input matrices are encrypted with homomorphic encryption. The result of HE MM is encrypted output matrix. HE MM is a fundamental kernel in many HE applications. Several algorithmic advancements [13]–[16] to reduce its computational complexity have been made. However, CPU-based implementations remain prohibitively slow, particularly for large matrix sizes. For example, multiplying two modestly sized matrices (e.g., 64×64) under a small HE parameter set can still take several tens of seconds [15], which is approximately $1000\times$ slower than the plaintext operation. This substantial performance gap highlights the need for hardware acceleration to make HE MM based applications practical.

FPGAs offer a unique advantage for accelerating HE MM due to their configurable on-chip memory hierarchy and customizable compute resources, enabling fine-grained datapath control to enhance on-chip data reuse. In addition, leading FPGA vendors also incorporate High Bandwidth Memory (HBM) to overcome traditional DRAM bandwidth limitation, making them well-suited for handling the high-throughput demands of encrypted data transfers.

The primary bottleneck in HE MM is homomorphic linear transformation (HLT), accounting for over 95% of the total runtime [13]. This inefficiency arises from two key challenges: (1) HLT requires multiple ciphertext rotations - one of the most computationally expensive operations in HE, and (2) it generates numerous intermediate ciphertexts during computation, leading to substantial off-chip memory traffic that limits the scalability of HE MM to larger matrices and HE parameter sets. To mitigate the first challenge, a pioneering work [17] introduced the *hoisting* technique to amortize certain sub-operations across multiple rotations, which has been

widely adopted in subsequent works [4], [18]–[20]. However, the second challenge remains unaddressed due to the lack of hardware-aware solutions. In this work, we address both challenges by designing a novel HLT datapath that integrates algorithmic optimizations to minimize off-chip memory traffic. Our key contributions are summarized as follows:

- We develop a cost model to analyze the on-chip memory requirements of HE MM for a given matrix size, shape, and HE parameter set. The analysis shows that HE MM is not scalable without explicit datapath optimizations due to substantial off-chip ciphertext traffic.
- We propose an on-chip memory-optimized datapath for HLT, the bottleneck in HE MM, through hardware-software co-design. The datapath significantly reduces off-chip memory traffic by fusing sub-operations and enabling fine-grained on-chip ciphertext reuse.
- Leveraging this datapath, we design FAME, the first accelerator tailored for HE MM. FAME achieves resource efficiency through reduced on-chip memory usage. The architecture can be configured to support diverse matrix sizes, shapes, and HE parameter sets.
- We implement FAME on Alveo U280. Experimental results show that FAME achieves $221\times$ speedup on average over state-of-the-art CPU implementations across diverse matrix sizes, shapes, and HE parameters, demonstrating its scalability and practicality for large-scale HE MM.

II. BACKGROUND

A. Threat Model

We assume a semi-honest (honest but curious) cloud service provider as previous works [12], [21]–[23]. The client sends the encrypted input matrix to the cloud server to perform HE MM. Importantly, the other input matrix is also encrypted, whether it comes from the client, a third-party model provider, or the server itself. This threat model differs from prior HE acceleration works [10]–[12], which typically assume only two parties (client and server) and a plaintext model. In contrast, our approach addresses scenarios with stronger privacy guarantees by ensuring both inputs remain encrypted throughout the computation.

B. Homomorphic Encryption (HE)

Among various HE schemes [24]–[27], we focus on the CKKS [27], which supports real-number computations, making it suitable for HE machine learning (ML) applications. In CKKS, a message vector $\mathbf{m}$ of $N/2$ numbers is first encoded into a plaintext (Pt) polynomial of degree $N-1$. The Pt is then encrypted into a ciphertext (Ct) $\llbracket \mathbf{m} \rrbracket := (\mathbf{a}, \mathbf{b})$, where both $\mathbf{a}$ and $\mathbf{b}$ are polynomials of degree $N-1$. Each Ct polynomial resides in the ring $\mathcal{R}_Q = \mathbb{Z}_Q[X]/(X^N + 1)$, meaning the polynomial coefficients are integers modulo Q .

1) *Residual Number System (RNS)*: The full modulus Q is typically hundreds to thousands of bits. To enable practical arithmetic, the RNS is used to decompose Q or Q_L into the product of the coprime moduli $Q_L = \prod_{i=0}^L q_i$, with each q_i fitting within a machine word. As a result, each ciphertext

polynomial is split into $L+1$ limbs, where each limb is a sub-polynomial modulo q_i : $[\mathbf{a}]_{Q_L} \mapsto [\mathbf{a}]_{q_0}, \dots, [\mathbf{a}]_{q_L}$. A freshly encrypted Ct (i.e., a fresh Ct) consists of $L+1$ such limbs.

2) *CKKS Operations*: Assume a Pt encodes a message vector $\mathbf{m}_p$. CKKS supports the following operations based on modular arithmetic over encrypted vectors.

- $\text{Add}(\llbracket \mathbf{m} \rrbracket, \llbracket \mathbf{m}' \rrbracket) \rightarrow \llbracket \mathbf{m} + \mathbf{m}' \rrbracket = (\mathbf{a} + \mathbf{a}', \mathbf{b} + \mathbf{b}')$
- $\text{Mult}(\llbracket \mathbf{m} \rrbracket, \llbracket \mathbf{m}' \rrbracket) \rightarrow \llbracket \mathbf{m} \odot \mathbf{m}' \rrbracket = (\mathbf{a} \odot \mathbf{a}', \mathbf{a} \odot \mathbf{b}' + \mathbf{b} \odot \mathbf{a}') + \text{KeySwitch}(\mathbf{b} \odot \mathbf{b}', \mathbf{evk}_{\text{mult}})$
- $\text{CMult}(\llbracket \mathbf{m} \rrbracket, \text{Pt}) \rightarrow \llbracket \mathbf{m} \odot \mathbf{m}_p \rrbracket = (\mathbf{a} \odot \text{Pt}, \mathbf{b} \odot \text{Pt})$
- $\text{Rot}(\llbracket \mathbf{m} \rrbracket, r) \rightarrow \llbracket \rho(\mathbf{m}; r) \rrbracket = \text{KeySwitch}(\psi_r(\mathbf{b}), \mathbf{evk}_{\text{rot}}^{(r)}) + (\psi_r(\mathbf{a}), \mathbf{0})$, where $\rho(\mathbf{m}; r)$ denotes a circular left rotation by r slots, and ψ_r is the automorphism (Automorph) on a polynomial (i.e., a coefficient permutation). For the index mapping of ψ_r , please refer to [28].

3) *Sub-operations*: Mult and Rot rely on the computationally intensive KeySwitch sub-operation, involving an inner product (KeyIP) between a Ct polynomial and an evaluation key $\mathbf{evk}$. The $\mathbf{evk}$ is a $2 \times \beta$ matrix of polynomials modulo PQ , where $P = \prod_{i=0}^{k-1} p_i$ is the auxiliary modulus. To perform KeyIP, the Ct polynomial is first decomposed (Decomp) into β digits, each comprising $\alpha := (L+1)/\beta$ limbs. Each digit is then raised in modulus (ModUp) from $Q_{\alpha+1}$ ($\alpha+1$ limbs) to PQ_L ($L+k+1$ limbs). The KeyIP is computed by multiplying the digit vector with each row of $\mathbf{evk}$ matrix and accumulating along the row. This results in two polynomials modulo PQ_L . Finally, the result is reverted to modulus Q_L using ModDown. For details of KeySwitch, we refer [4], [29], [30].

Number Theoretic Transform (NTT), a finite-field variant of the Discrete Fourier Transform [31], enables element-wise polynomial multiplication in the evaluation domain. To avoid unnecessary NTTs and its inversion iNTTs, polynomials typically remain in the evaluation domain [28], [32], [33]. However, ModUp and ModDown require base conversion (BaseConv) to change the modulus (number of limbs), which cannot be performed in the evaluation domain. Therefore, iNTT is applied beforehand, and NTT is applied afterward.

4) *Ct Levels and Rescaling*: Each Mult or CMult is followed by a Rescale, which is a special case of ModDown, dropping the last limb (q_L) of the Ct polynomial [28], [32]. After L such operations, only the base limb (q_0) remains, and further multiplications are not allowed. Thus, L defines the maximum (or initial) Ct level, which supports an evaluation circuit with a computation depth of L . We use ℓ to denote the current Ct level, replacing L when the Ct is not fresh.

C. Secure Matrix Multiplication with HE

In this work, we consider a general method for HE MM with arbitrary matrix shapes. Given $A_{m \times l} \times B_{l \times n}$, state-of-the-art (SOTA) approaches [13]–[16] rely on element-wise multiplication, where each matrix is encrypted into a Ct. We follow the notations from [16], as shown in Eq. 1.

$$A_{m \times l} \times B_{l \times n} = \sum_{k=0}^{l-1} (\epsilon^k \circ \sigma(A)) \odot (\omega^k \circ \tau(B)) \quad (1)$$

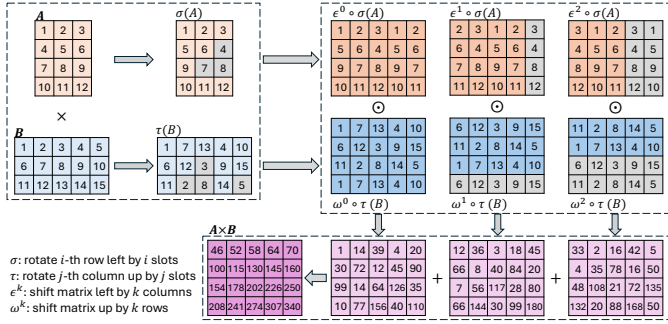


Fig. 1. Element-wise MM with an example of $m=4, l=3, n=5$

where $\circ$ denotes composition. The four matrix transformation operators are defined as:

$$\sigma(A)_{i,j} = A_{i,[i+j]_l}, \quad 0 \leq i < m, 0 \leq j < l \quad (2)$$

$$\tau(B)_{i,j} = B_{[i+j]_l,j}, \quad 0 \leq i < l, 0 \leq j < n \quad (3)$$

$$\epsilon^k(A)_{i,j} = A_{i,[j+k]_l}, \quad 0 \leq i < m, 0 \leq j < n \quad (4)$$

$$\omega^k(B)_{i,j} = B_{[i+k]_l,j}, \quad 0 \leq i < m, 0 \leq j < n \quad (5)$$

Fig. 1 illustrates an example of $A_{4 \times 3} \times B_{3 \times 5}$. With HE, each input matrix is flattened into a vector $\mathbf{m}$. Now, the four matrix transformations can be expressed as linear transformations $\mathbf{m} \rightarrow U \cdot \mathbf{m}$, where U is the corresponding transformation matrix. Assuming both A and B are flattened in the column-major order, the four transformation matrices are:

$$U_{i+j \cdot m, h}^\sigma = \begin{cases} 1 & \text{if } h = i + [i+j]_l \cdot m \\ 0 & \text{otherwise} \end{cases} \quad (6)$$

$$U_{i+j \cdot l, h}^\tau = \begin{cases} 1 & \text{if } h = [i+j]_l + j \cdot l \\ 0 & \text{otherwise} \end{cases} \quad (7)$$

$$U_{i,j}^{\epsilon^k} = \begin{cases} 1 & \text{if } j = [k \cdot m + i]_{m \cdot l} \\ 0 & \text{otherwise} \end{cases} \quad (8)$$

$$U_{i,j}^{\omega^k} = \begin{cases} 1 & \text{if } j = [k + [i]_m]_l + [i/m] \cdot l \\ 0 & \text{otherwise} \end{cases} \quad (9)$$

The four transformation matrices are sparse, each with multiple non-zero diagonals. For $U \in \mathbb{Z}_{x \times y}$ and $-x < z < y$, the z -th diagonal is defined as:

$$\mathbf{u}_z = \begin{cases} (U_{0,z}, U_{1,z+1}, \dots, U_{y-1-z,y-1}, 0, \dots, 0) & \text{if } z > 0 \\ (0, \dots, 0, U_{-z,0}, U_{1-z,1}, \dots, U_{x-1,x-1-z}) & \text{if } z < 0 \end{cases} \quad (10)$$

If U has d non-zero diagonals indexed by $z_0, \dots, z_{d-1}$, the matrix-vector product can be computed using a combination of rotation and element-wise multiplication:

$$U \cdot \mathbf{m} = \sum_{0 \leq t < d} (\mathbf{u}_{z_t} \odot \rho(\mathbf{m}; z_t)) \quad (11)$$

Since both A and B are encrypted, the linear transformations must be performed homomorphically, referred to as HLT shown in Algorithm 1. Each HLT includes CMult, which requires a Rescale at the end, reducing the Ct level by one. The MM in Eq. 1 is described in Algorithm 2 with HE.

Algorithm 1 Homomorphic Linear Transformation

Input: Encrypted matrix: Ct, Pre-computed non-zero Pt diagonals of U and their indices: $\{\mathbf{u}_{z_t}, z_t\}$

Output: Ct' (initialized as zero) $\leftarrow$ HLT(Ct, U)

- 1: **for** $t = 0$ to $d - 1$ **do**
- 2: Ct' $\leftarrow$ Add(Ct', CMult(Rot(Ct; z_t); $\mathbf{u}_{z_t}$))
- 3: **end for**
- 4: **return** Rescale(Ct')

Algorithm 2 Homomorphic Encrypted Matrix Multiplication

Input: Ct_A, Ct_B, $U^\sigma, U^\tau, U^{\epsilon^k}, U^{\omega^k}$

- 1: Ct_{A(0)} $\leftarrow$ HLT(Ct_A, U^σ)
- 2: Ct_{B(0)} $\leftarrow$ HLT(Ct_B, U^τ)
- 3: **for** $k = 0$ to $l - 1$ **do**
- 4: Ct_{A(k+1)} $\leftarrow$ HLT(Ct_{A(0)}, U^{ϵ^k})
- 5: Ct_{B(k+1)} $\leftarrow$ HLT(Ct_{B(0)}, U^{ω^k})
- 6: Ct_{AB} $\leftarrow$ Add(Ct_{AB}, Mult(Ct_{A(k+1)}, Ct_{B(k+1)}))
- 7: **end for**
- 8: **return** Ct_{AB}

III. COMPLEXITY ANALYSIS AND COST MODEL

In this section, we analyze the computational complexity of HE MM and introduce a cost model for estimating the on-chip memory required for Ct in performing HE MM.

A. Complexity Analysis

Algorithm 2 has two steps: Step 1 (Lines 1–2): Generate Ct_{A(0)} and Ct_{B(0)}. Step 2 (Lines 3–6): For each $k = 0$ to $l - 1$, generate Ct_{A(k+1)} and Ct_{B(k+1)}, and perform a multiply-accumulate operation to produce the final encrypted result.

In total, HE MM requires $2 \cdot (l + 1)$ HLT operations. However, the cost of each HLT varies with the number of non-zero diagonals (d) in the associated transformation matrix, as this directly impacts the number of required Rot. The values of d for each type of transformation matrix are as follows [16]:

$$d_{U^\sigma} = 2 \cdot \min(m, l) - 1 \quad (12)$$

$$d_{U^\tau} = 2 \cdot \min(n, l) - 1 \quad (13)$$

$$d_{U^{\epsilon^k}} = \lfloor \frac{n}{l} \rfloor + 1 \quad (14)$$

$$d_{U^{\omega^k}} = \begin{cases} 2 & \text{if } m = l \\ n \cdot (\lfloor \frac{m}{l} \rfloor + 2) & \text{otherwise} \end{cases} \quad (15)$$

The complexity of general HE MM is summarized in Table I. Each HLT reduces the Ct level by one due to the required Rescale. Additionally, the Mult operation in Step 2 incurs one more level reduction. Therefore, evaluating a single HE MM requires the Ct to have at least four levels, i.e., $L \geq 4$.

B. Cost Model

Given the size of input matrices and HE parameters, the model estimates the on-chip memory needed to store all intermediate Cts on the chip while performing HE MM. Since each Ct encrypts a vector of size $N/2$, the minimal polynomial

TABLE I
COMPLEXITY AND REQUIRED DEPTH OF GENERAL HE MM

	Add	Mult	CMult	Rot	Depth*
Step 1	ϕ	0	ϕ	ϕ	1
Step 2	$\zeta + l$	l	ζ	ζ	2
Total	$\phi + \zeta + l$	l	$\phi + \zeta$	$\phi + \zeta$	3

$\phi = d_{U^\sigma} + d_{U^\tau}; \zeta = l \cdot (d_{U^{\epsilon^k}} + d_{U^{\omega^k}})$
* Consumed Ct levels

degree N must be large enough to encode each matrix ($A_{m \times l}$ and $B_{l \times n}$). Thus, N is given by:

$$N = \max(2^{\lceil \log_2(2ml) \rceil}, 2^{\lceil \log_2(2nl) \rceil}), \quad (16)$$

1) *Data Sizes*: Let $\mathcal{B}_{\text{coeff}}$ denote the size (in bytes) of a single limb coefficient. Then, the size of one limb is given by $\mathcal{B}_{\text{limb}} = N \cdot \mathcal{B}_{\text{coeff}}$. The size of a Ct at level ℓ is:

$$\mathcal{B}_{\text{Ct}}^\ell = 2 \cdot (\ell + 1) \cdot \mathcal{B}_{\text{limb}} = 2 \cdot N \cdot \log Q_\ell / 8 \quad (17)$$

Each encoded Pt has the size of $\mathcal{B}_{\text{limb}}$. The HLT requires Pt diagonals of U , so the total Pt size is proportional to the number of non-zero diagonals. Additionally, each Rot and Mult operation requires an *evk*, with its size defined in Eq. 18. Moreover, the NTT and iNTT require additional twiddle factors, which are proportional to limb count.

$$\mathcal{B}_{\text{evk}}^\ell = 2 \cdot \beta \cdot (\ell + k + 1) \cdot \mathcal{B}_{\text{limb}} \quad (18)$$

2) *On-chip Memory Requirement*: As Pt diagonals, *evk*, and twiddle factors are read-only after being fetched, we consider the on-chip memory requirement for Ct to explore on-chip Ct reuse. In addition to storing two Cts of input matrices, we need to consider the intermediate Cts generated during HE MM. For simplicity, we assume all Cts are fresh (at maximum level L) and ignore Ct level reductions in our analysis, providing an upper bound on memory requirement.

The primary bottleneck is the KeySwitch, required by both Rot and Mult, with Rot being heavily executed in HLT. During KeySwitch, one input Ct polynomial expands from $0.5 \cdot \mathcal{B}_{\text{Ct}}^L$ to $0.5 \cdot \beta \cdot \mathcal{B}_{\text{Ct}}^{L+k+1}$ for KeyIP and produces an output Ct of size $\mathcal{B}_{\text{Ct}}^L$. Thus, the required on-chip memory for KeySwitch to eliminate off-chip memory traffic of intermediate Ct is:

$$\mathcal{M}_{\text{KeySwitch}}^{\text{Ct}} = \mathcal{B}_{\text{Ct}}^L + 0.5 \cdot \beta \cdot \mathcal{B}_{\text{Ct}}^{L+k+1} \quad (19)$$

Each Rot is applied to a Ct consisting of two polynomials. After applying Automorph, one polynomial, $\psi(\mathbf{b})$ is passed to KeySwitch, requiring $\mathcal{M}_{\text{KeySwitch}}^{\text{Ct}}$. In addition, both the original Ct ($\mathbf{a}, \mathbf{b}$) and the $\psi(\mathbf{a})$ must be retained. Therefore, the total on-chip memory required for a single Rot is

$$\mathcal{M}_{\text{Rot}}^{\text{Ct}} = \mathcal{M}_{\text{KeySwitch}}^{\text{Ct}} + 1.5 \cdot \mathcal{B}_{\text{Ct}}^L \quad (20)$$

For HLTs in Step 1, only one input Ct buffer is required, since Ct_A and Ct_B are not reused. For HLTs in Step 2, two input Ct buffers are required, as ($\text{Ct}_{A(0)}$ and $\text{Ct}_{B(0)}$) are reused across multiple iterations. In both steps, two output buffers are needed to store the resulting Cts. The multiply-accumulate in

HLT can be computed in place. Thus, the on-chip memory requirements for HLT in each step are:

$$\mathcal{M}_{\text{HLT.s1}}^{\text{Ct}} = \mathcal{M}_{\text{Rot}}^{\text{Ct}} + 3 \cdot \mathcal{B}_{\text{Ct}}^L \quad (21)$$

$$\mathcal{M}_{\text{HLT.s2}}^{\text{Ct}} = \mathcal{M}_{\text{Rot}}^{\text{Ct}} + 4 \cdot \mathcal{B}_{\text{Ct}}^L \quad (22)$$

The memory overhead for the Mult in Step 2 is minimal compared to HLT. The in-place Add operation requires an additional buffer to store the accumulated result Ct_{AB} . Therefore, the total on-chip memory requirement for storing Ct during HE MM is given in Eq. 23. Given the matrix dimensions and HE parameter set, all intermediate Cts can be stored on-chip if the available on-chip memory exceeds this requirement.

$$\mathcal{M}_{\text{HE-MM}}^{\text{Ct}} = \mathcal{M}_{\text{HLT.s2}}^{\text{Ct}} + \mathcal{B}_{\text{Ct}}^L \quad (23)$$

3) *Example Analysis*: In practice, encrypting a modest-sized matrix (e.g., 64×64) requires a relatively small HE parameter set with a low polynomial degree, e.g., $N = 2^{13}$. Under such configurations (e.g., Set-A in Table II, Sec. VI-A), each Ct occupies only 0.43 MB, resulting in a manageable total on-chip memory requirement of approximately 3.6 MB. This can be comfortably accommodated by SOTA CPUs with a last-level cache (LLC) of 32 MB per die [34]. However, this small parameter set (Set-A) has limited practical utility. Real-world applications generally require larger HE parameter sets (e.g., Set-B and Set-C in Table II) for three main reasons: (1) larger matrices cannot fit within small polynomial degrees; (2) deeper evaluation circuits demand higher Ct levels (L) to support consecutive MM operations; and (3) stronger security levels, e.g., 128-bit security, may require larger parameter sizes.

For example, Set-B increases each Ct size to 6.7 MB, elevating the total on-chip memory requirement to about 61 MB for HE MM. Even more practical parameter sets, like Set-C, further expand each Ct to 27 MB, resulting in a total on-chip memory demand of approximately 255 MB. At this scale, storing two encrypted matrices simultaneously on-chip becomes impractical. Even the requirement for a single KeySwitch operation ($\mathcal{M}_{\text{KeySwitch}}^{\text{Ct}}$) can exceed available resources. Consequently, large parameter sets lead to considerable off-chip memory traffic. Without explicit datapath and memory control, each KeySwitch can involve hundreds of MBs of off-chip memory access due to repeated reads and writes Cts across sub-operations. Given that each Rot operation invokes a KeySwitch and each HLT typically involves numerous rotations (e.g., 127 for $m = l = n = 64$), this can result in tens or even hundreds of GB of off-chip Ct transfers per HLT invocation, creating a severe performance bottleneck.

C. Key Takeaway

Our analysis highlights that significant off-chip Ct traffic severely limits the scalability of HE MM. The prohibitive memory demands associated with practical HE parameter sets make traditional CPU-based methods infeasible for large-scale HE MM. To overcome this memory bottleneck, Section IV introduces hardware-aware optimizations designed explicitly to minimize off-chip memory accesses and enhance scalability.

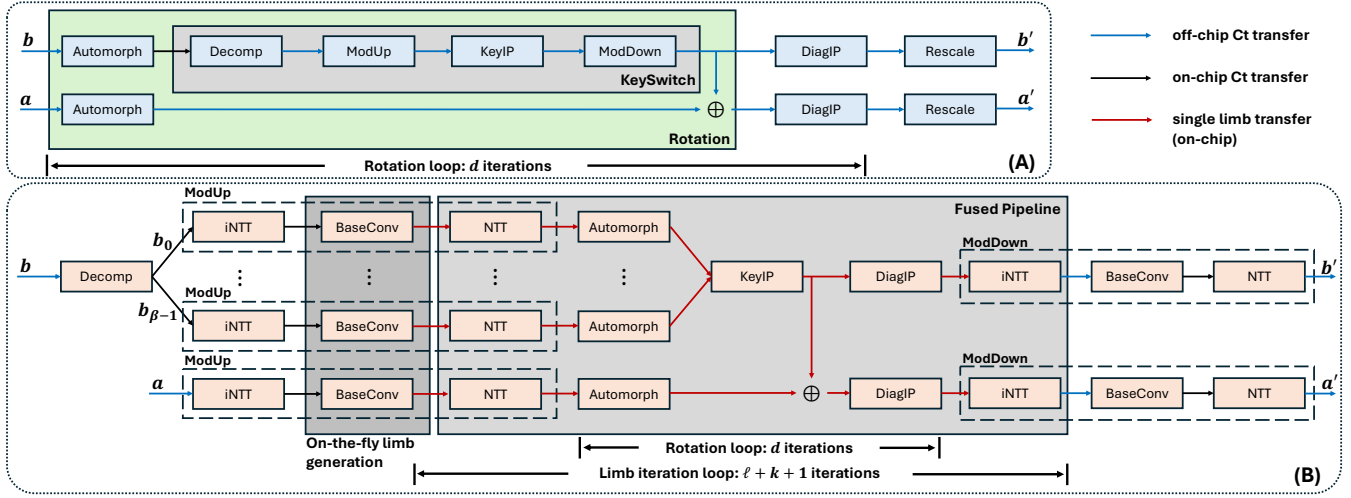


Fig. 2. For a practical parameter set (i.e., Set-C): (A) Baseline HLT design with coarse-grained rotation loop and full ciphertext-level datapath, incurring high DRAM traffic. (B) Proposed memory-optimized HLT (MO-HLT), an architecture-algorithm co-designed solution, which enables on-the-fly limb generation and sub-operation fusion across NTT, Automorph, KeyIP, DiagIP, and iNTT, drastically reducing SRAM requirement and DRAM access.

Algorithm 3 HLT Represented with Sub-operations

Input: $\llbracket m \rrbracket := (a, b) \in \mathcal{R}_{Q_\ell}^2$; Pre-computed Pt diagonals of U and their indices: $\{u_{z_t}, z_t\}$

Output: $\llbracket m' \rrbracket := (a', b')$ initialized as 0

```

1:  $(b_j)_{j \in [0, \beta]} \leftarrow \text{Decomp}(b)$ 
2:  $(\hat{a}, \hat{b}_j) \leftarrow (\text{ModUp}(a), \text{ModUp}(b_j))$  for  $j \in [0, \beta]$ 
3: for  $t$  from 0 to  $d - 1$  do
4:    $\hat{a}_{\text{rot}} \leftarrow \text{Automorph}(\hat{a}, z_t)$ 
5:    $\hat{b}_{\text{rot}}^{(j)} \leftarrow \text{Automorph}(\hat{b}_j, z_t)$  for  $j \in [0, \beta]$ 
6:    $(\hat{u}, \hat{v}) \leftarrow \text{KeyIP}(\hat{b}_{\text{rot}}^{(j)}, \text{evk}_{\text{rot}}^{(z_t)})$ 
7:    $(a', b') += u_{z_t} \cdot (\hat{u}, \hat{v} + \hat{b}_{\text{rot}})$   $\triangleright$  DiagIP
8: end for
9:  $\llbracket m' \rrbracket \leftarrow (\text{ModDown}(\hat{a}'), \text{ModDown}(\hat{b}'))$ 
10: return  $\text{Rescale}(\llbracket m' \rrbracket)$   $\triangleright \mathcal{R}_{Q_{\ell-1}}^2$ 

```

IV. MEMORY-OPTIMIZED HLT DATAPATH

Motivated by the off-chip traffic bottleneck identified in Sec. III-B2, we propose a memory-optimized datapath for HLT (MO-HLT) that reduces on-chip memory requirement to minimize off-chip Ct traffic. Our approach begins with two algorithmic optimizations shown in Algorithm 3. The multiply-accumulate step in Line 7 is denoted as DiagIP (diagonal inner product), encapsulating the CMult and Add operations. First, sub-operations Decomp, ModUp, and ModDown are moved outside the rotation loop, which is known as the *hoisting* technique [4], [35]. These hoisted sub-operations are shared across multiple KeySwitch calls and thus executed only once rather than per Rot. Second, the Rescale is merged with ModDown, enabling a direct modulus reduction from PQ_ℓ to $Q_{\ell-1}$, bypassing the intermediate Q_ℓ [4], [36].

Integrating algorithmic optimizations above, Fig. 2(B) illustrates the proposed MO-HLT, an architecture-algorithm co-designed solution featuring two memory-centric optimizations: (1) *fine-grained datapath via sub-operation fusion*, and (2) *on-*

the-fly limb generation. These collectively maximize on-chip Ct reuse and minimize off-chip data traffic.

The first optimization, *fine-grained datapath via sub-operation fusion*, improves on-chip Ct reuse by operating at the limb level rather than the full Ct level. As discussed in Sec. III-B2, full Ct-level reuse is infeasible for large matrices with large HE parameters, due to limited on-chip memory (e.g., 43 MB of Alveo U280). Instead, we enable reuse across sub-operations by allowing each Ct limb to proceed independently through the datapath, without waiting for all Ct limbs to complete the current stage. This approach significantly reduces both on-chip memory demand and off-chip traffic. Sub-operation fusion is applied within the rotation loop of Algorithm 3 across Automorph, KeyIP, and DiagIP.

The second optimization, *on-the-fly limb generation*, further reduces on-chip memory pressure by enabling immediate pipelined processing. During ModUp, each newly generated limb from BaseConv is directly forwarded to NTT without waiting for the full extended Ct ($\ell + k + 1$ limbs) to be assembled. This allows the NTT in ModUp to be seamlessly fused into the pipeline. Similarly, the iNTT in ModDown is fused with the preceding DiagIP, reducing off-chip Ct traffic.

Together, these optimizations yield the final fused sub-operation set: NTT, Automorph, KeyIP, DiagIP, iNTT. As shown in Fig. 2(B), a key architectural change is the reordering of the Ct processing loops. In prior coarse-grained approaches shown in Fig. 2(A), the outer loop iterates over rotations, and each Ct proceeds through sub-operations collectively at the full-Ct level (i.e., all limbs are processed together for each sub-operation). In contrast, our fine-grained datapath with sub-operation fusion reverses this order: the limb iteration becomes the outer loop, and the rotation loop moves inside. This enables pipelined execution and allows each limb to proceed through the sub-operations independently, improving on-chip Ct reuse.

The proposed MO-HLT reduces on-chip memory require-

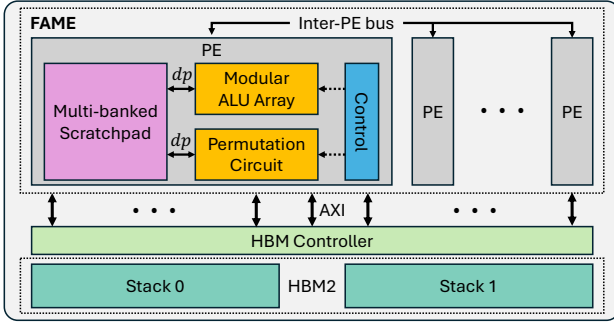


Fig. 3. The overall system architecture on FPGA

ment by storing only one Ct and $(\beta + 1)$ intermediate limbs to support pipelined processing:

$$M_{\text{MO-HLT}}^{\text{Ct}} = \mathcal{B}_{\text{Ct}}^L + (\beta + 1) \cdot \mathcal{B}_{\text{limb}} \quad (24)$$

For Set-C, MO-HLT requires about 29 MB, easily fitting within typical CPU cache capacities. MO-HLT only reads the input Ct and stores the output Ct, with trivial additional intermediate off-chip Ct transfers during ModDown. The total off-chip traffic for each HLT is limited to roughly a hundred MB, which is several orders of magnitude smaller than prior CPU-based approaches, making the design highly scalable. For even larger HE parameter sets (e.g., $N = 2^{17}$, where a Ct can exceed 60 MB and cannot fit on-chip), MO-HLT remains effective. In such cases, only the unfused sub-operations incur off-chip traffic, while fused sub-operations still benefit from pipelined limb reuse. Overall, MO-HLT significantly reduces both off-chip traffic and on-chip memory demand, effectively addressing the memory bottleneck in practical, large-scale HE MM. Therefore, it is well-suited for on-chip memory-constrained platforms such as CPUs, GPUs, and FPGAs.

V. HE MM ACCELERATOR - FAME

A. Overall Architecture

Fig. 3 illustrates the system architecture, which consists of the FAME accelerator and two HBM stacks on Alveo U280. To interface with the 32 AXI ports of the HBM, we instantiate 32 asynchronous FIFO pairs for concurrent write and read operations. Each FIFO has a data width of 256 bits, aligning with the bit-width of the AXI interface. FAME is composed of multiple processing elements (PEs), each designed with a fully pipelined architecture to improve resource utilization and maximize throughput. An inter-PE bus, with a width equal to the data parallelism (dp), enables communication between PEs for polynomial transfer. Prior to computation, data from the read FIFOs is packed into dp -wide data lanes and streamed into the multi-banked scratchpad memory within each PE. FAME features a highly configurable architecture. Key design parameters—including the number of PEs, dp within each PE, and the scratchpad size per PE—can be tuned to accommodate various matrix sizes and HE parameter sets, while adapting to the available FPGA resources.

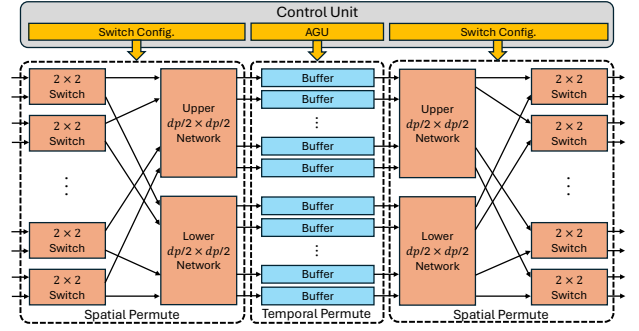


Fig. 4. Fully pipelined permutation circuit (dp -to- dp)

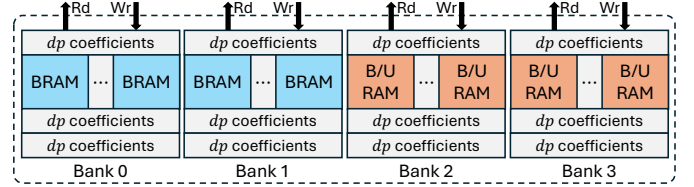


Fig. 5. Scratchpad memory organization with BRAM or URAM banks

B. PE Architecture

Each PE comprises three key components: (1) a modular arithmetic logic unit (ALU) array, (2) a configurable multi-banked scratchpad, and (3) a permutation circuit.

1) *Modular ALU Array*: Polynomial computations are reduced to basic modular operations (addition, subtraction, and multiplication). Accordingly, each modular ALU array contains dp modular ALUs, each consisting of one modular adder (configurable for addition or subtraction) and one modular multiplier. The modular multiplier is implemented with a pipelined Barrett reduction design [37]. We adopt 54-bit RNS prime integers ($\log q$) for efficient utilization of DSPs [38].

2) *Permutation Circuit*: The permutation circuit, shown in Fig. 4, handles two permutation tasks in HE: (i) radix-2 butterfly permutations for (i)NTT and (ii) index mapping (ψ) of Automorph. Our design employs a streaming permutation network (SPN) [39], enabling arbitrary parallel coefficient permutations without resorting to a costly crossbar. We redesign its control logic for HE to support Automorph. The circuit includes three subnetworks: two spatial permutation networks and one temporal permutation network. Each spatial network consists of $\log dp$ stages with $(dp/2) 2 \times 2$ switches to achieve complete dp -to- dp connectivity. The address generation unit (AGU) coordinates the temporal permutation by controlling dp dual-port buffers. Each buffer stores N/dp coefficients, facilitating coefficient permutations across different processing cycles. The fully pipelined permutation circuit achieves a throughput of dp coefficients per cycle.

3) *Multi-banked Scratchpad*: The scratchpad memory is organized into four dual-port banks, each with a width of dp , as shown in Fig. 5. These banks store various polynomials, including Ct, Pt, evk, and twiddle factors. The scratchpad supports up to four concurrent read and write requests from the modular ALU array, the permutation circuit, and the HBM

TABLE II
HE PARAMETER SETS USED FOR EVALUATION

	N	$\log Q$	L^*	k	β	λ
Set-A	2^{13}	218	4	1	1	80
Set-B	2^{15}	855	15	8	2	128
Set-C	2^{16}	1693	31	12	3	128

* Fresh Ct levels; supported computation depth

interface. To accommodate a large matrix encrypted with a large Ct, two of the banks can be configured and implemented using URAMs. The depth of each bank is also configurable, allowing the total scratchpad size to be tailored based on the on-chip memory cost model for MO-HLT. This flexibility enables support for a wide range of matrix sizes and HE parameter sets.

VI. EVALUATION

A. Experimental Setup

We implement FAME using Verilog HDL and map it on Alveo U280. The FPGA has 1,304K LUTs, 2,607K FFs, 43 MB on-chip SRAM, and 9,024 DSPs. We perform synthesis, place-and-route using Vivado 2023.1. The results are reported after place-and-route. We run RTL simulations to report latency. Latency is defined as the duration from loading the input encrypted matrices from the HBM to storing the output encrypted matrix back into the HBM.

Since prior CPU-based implementations use different schemes, libraries, and platforms, we reimplement several representative approaches [13]–[16] using the CKKS scheme with Pyfhel [40] for fair and consistent benchmarking. Experiments are conducted on a high-performance server CPU (Intel Xeon Gold 6326) with 32 cores (64 threads), a 24 MB cache, and running at 2.9 GHz. Specifically, the following baseline approaches are implemented for comparison:

- E2DM-S [13]: A method targeting square MM. For general MM $A_{m \times l} \times B_{l \times n}$, matrices are padded to square dimensions $s \times s$, where $s = \max\{m, l, n\}$.
- E2DM-R [13]: An optimized variant of E2DM-S for rectangular MM of the form $A_{m \times l} \times B_{l \times l}$ or $A_{l \times l} \times B_{l \times n}$.
- Huang et al. [15]: A pioneering approach supporting arbitrary matrix shapes.
- HEGMM-En [16]: A SOTA approach for arbitrary matrix shapes, with improved performance when $m = \min\{m, l, n\}$ or $n = \min\{m, l, n\}$.

1) *HE Parameter Sets and Benchmarks*: We adopt three HE parameter sets (Table II) to support varying matrix sizes. All ensure at least 80-bit security (λ), with Set-B&C achieving 128-bit security. Notably, Set-B&C offer multiple Ct levels, enabling consecutive MMs and supporting real-world applications with deep evaluation circuits.

We target experiments for general MM with HE. Therefore, we select various matrix shapes for experiments, categorized into four types: Type-I: $n = \min\{m, l, n\}$; Type-II: $l = \min\{m, l, n\}$; Type-III: $m = \min\{m, l, n\}$; Type-IV: square. Our MM benchmarks (denoted as m - l - n) along with their corresponding HE parameter sets are listed in Table III.

TABLE III
MM BENCHMARKS WITH DIVERSE SIZES AND SHAPES

	Set-A	Set-B	Set-C
Type-I	64-64-16	128-128-16	160-160-16
Type-II	64-16-64	128-16-128	160-16-160
Type-III	16-64-64	16-128-128	16-160-160
Type-IV	64-64-64	128-128-128	160-160-160

TABLE IV
FAME CONFIGURATIONS AND ACHIEVED FREQUENCY

	FAME-S	FAME-M	FAME-L
# of PEs	2	2	1
# of Lanes per PE (dp)	128	128	256
Scratchpad Size per PE	864 KB	7.6 MB	30.4 MB
Frequency (MHz)	350	350	300

2) *FAME Configurations*: We adopt different accelerator configurations tailored to each HE parameter set: FAME-S (Small), FAME-M (Medium), and FAME-L (Large), as detailed in Table IV. FAME-S is used with Set-A for small matrix sizes, FAME-M with Set-B, and FAME-L with Set-C.

Each configuration features a different scratchpad size to support data reuse for MO-HLT. Under constrained DSP resources, both FAME-S and FAME-M employ 2 PEs, each with a dp of 128. In these configurations, each PE handles the HLTs for one of the input matrices (A or B). An inter-PE bus is used to transfer intermediate Ct limbs between PEs after each HLT in Step 2, enabling the accumulation of the final result. For Set-C, the on-chip memory is not sufficient to store both A and B simultaneously. Therefore, FAME-L adopts a single-PE configuration with $dp = 256$, processing the inputs sequentially while maintaining throughput.

B. Performance

Fig. 6 presents the evaluation results for each CPU-based approach alongside the speedup achieved by FAME. The reported speedup is computed relative to the best-performing CPU baseline for each MM benchmark. The latency of the best CPU implementation is annotated above its corresponding bar.

FAME consistently achieves higher speedups on Type-I and Type-IV benchmarks compared to Type-II and Type-III across all parameter sets. This trend can be attributed to two key factors: First, FAME exhibits lower latency on Type-I and Type-IV benchmarks because they share the property $m = l$, which results in $d_{U^{\omega^k}} = 2$. Consequently, only two Rot operations are required for each HLT involving U^{ω^k} in Step 2. Second, CPU-based methods such as E2DM-R [13] and HEGMM-En [16] implement specialized optimizations for rectangular matrix shapes. In contrast, FAME adopts a unified and general method for handling various matrix shapes, without relying on shape-specific optimizations. As a result, the relative speedup achieved by FAME is lower on Type-II and Type-III benchmarks due to the increased number of rotation operations required. Among all types, Type-IV consistently yields the highest speedup for FAME across all HE parameter sets, offering a fair comparison against CPU-based approaches under balanced matrix dimensions.

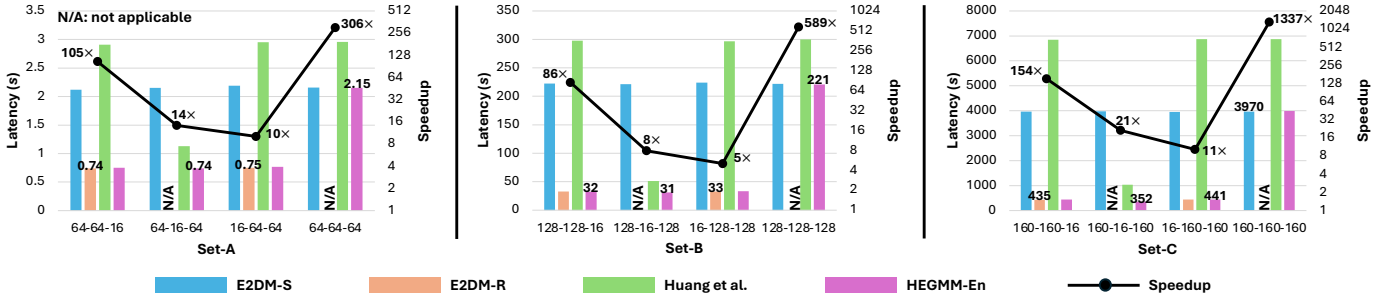


Fig. 6. HE MM latency of CPU-based approaches and achieved speedups by FAME over the best CPU result (shown above the corresponding bar)

TABLE V
RESOURCE CONSUMPTION OF FAME CONFIGURATIONS

	DSP	BRAM	URAM	LUT	FF
FAME-S	5,376	1,024	0	636k	998k
FAME-M	5,376	640	192	701k	1,147k
FAME-L	5,376	3,328	672	803k	1,660k

FAME achieves the highest overall speedups in Set-C, compared to the other parameter sets. This aligns with our expectation that the proposed MO-HLT datapath offers greater benefits to reduce off-chip Ct traffic for large benchmarks with a large HE parameter set. Notably, it completes the MM of two 160×160 encrypted matrices in about three seconds—1337 $\times$ faster than the best CPU implementation. On average, FAME delivers a 221 $\times$ speedup across all benchmarks, implying it can perform roughly 221 consecutive HE MMs in the time a CPU takes to complete just one, assuming sufficient Ct levels are available. These results highlight FAME’s ability to scale to large-scale consecutive MM efficiently, demonstrating its practicality for real-world, fully encrypted applications.

C. Resource Utilization

Table V summarizes the resource usage of each FAME configuration. As we scale from FAME-S to FAME-L, memory consumption increases accordingly to support larger matrices and HE parameters. In detail, each modular ALU uses 21 DSPs. In addition to the scratchpad, BRAMs are also allocated for temporal buffers in the permutation circuit.

D. Discussion on Scalability

This paper considers HE MM with each input matrix encrypted into a single Ct. Thus, the largest matrix size is limited by the Ct polynomial degree. We demonstrate the scalability of adopting large HE parameters used in practical applications, making it amenable to consecutive MM requiring multiple Ct levels. We evaluate FAME on the largest matrix that fits within a single Ct. For even larger matrices, the block MM approach encrypting a matrix with multiple Cts is required.

VII. RELATED WORK

HE Acceleration A wide range of HE acceleration efforts have been developed on platforms such as CPUs/GPUs [32], [41]–[43], FPGAs [44]–[46], and ASICs [8], [28], [36], targeting for various end-to-end HE applications, such as

sorting [47], image classification [48], and natural language processing [49]. However, existing accelerators typically rely on plaintext models, encrypting only the inputs while leaving the model unencrypted. To the best of our knowledge, FAME is the first accelerator supporting computations with both inputs encrypted, thus meeting stricter security requirements.

HE MM is a fundamental kernel for fully encrypted applications, where both input matrices are encrypted. Existing approaches [13]–[16], [50] target CPU-based implementations, developing algorithmic optimizations tailored to specific matrix shapes to reduce computational complexity. However, these implementations typically support only small HE parameter sets with limited Ct levels, restricting their applicability to deep evaluation circuits and stronger security requirements. Due to the lack of scalability in CPU-based solutions, existing methods have struggled to handle HE MM with large HE parameter sets, which limits their applicability to large-scale and consecutive MM workloads. Therefore, we introduce FAME, the first hardware accelerator specifically designed for HE MM. FAME enables efficient and scalable computation under practical configuration and large benchmarks, adopting a unified method applicable across diverse matrix shapes.

VIII. CONCLUSION

In this work, we presented FAME, the first FPGA-based accelerator for HE MM with both input matrices encrypted. To overcome the performance and scalability limitations of prior CPU-based solutions, FAME leverages the proposed MO-HLT datapath, significantly reducing off-chip traffic through algorithm-architecture co-design. FAME achieves resource efficiency by minimizing on-chip memory usage. Its configurable architecture supports a wide range of HE parameter sets and matrix dimensions. Implemented on Alveo U280, FAME demonstrates scalable, low-latency HE MM, paving the way for fully encrypted applications. Future work will explore extending FAME to the block MM approach and real-world workloads requiring consecutive MM. We will also extend FAME with support for shape-specific optimizations.

ACKNOWLEDGMENT

This work is supported by the U.S. National Science Foundation (NSF) under grants CSSI-2311870 and SaTC-2104264. Equipment and support from AMD AECG are greatly appreciated. **Distribution Statement A:** Approved for public release. Distribution is unlimited.

REFERENCES

- [1] B. Varghese and R. Buyya, "Next generation cloud computing: New trends and research directions," *Future Generation Computer Systems*, vol. 79, pp. 849–861, 2018.
- [2] T. Vasiljeva, S. Shaikhulina, and K. Kreslins, "Cloud computing: Business perspectives, benefits and challenges for small and medium enterprises (case of latvia)," *Procedia Engineering*, vol. 178, pp. 443–451, 2017.
- [3] C. S. Alliance, "Security guidance for critical areas of focus in cloud computing," 2009. [Online]. Available: <http://www.cloudsecurityalliance.org>
- [4] L. de Castro, R. Agrawal, R. Yazicigil, A. Chandrakasan, V. Vaikuntanathan, C. Juvekar, and A. Joshi, "Does fully homomorphic encryption need compute acceleration?" *arXiv preprint arXiv:2112.06396*, 2021.
- [5] B. Reagen, W.-S. Choi, Y. Ko, V. T. Lee, H.-H. S. Lee, G.-Y. Wei, and D. Brooks, "Cheetah: Optimizing and accelerating homomorphic encryption for private inference," in *2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2021, pp. 26–39.
- [6] A. Ebel, K. Garimella, and B. Reagen, "Orion: A fully homomorphic encryption compiler for private deep neural network inference," *arXiv preprint arXiv:2311.03470*, 2023.
- [7] Y. Zhu, X. Wang, L. Ju, and S. Guo, "Fxbenn: Fpga-based acceleration framework for homomorphic encrypted cnn inference," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 896–907.
- [8] N. Samardzic, A. Feldmann, A. Krastev, N. Manohar, N. Genise, S. Devadas, K. Eldefrawy, C. Peikert, and D. Sanchez, "Craterlake: a hardware accelerator for efficient unbounded computation on encrypted data," in *Proceedings of the 49th Annual International Symposium on Computer Architecture*, 2022, pp. 173–187.
- [9] T. Ye, S. R. Kuppannagari, R. Kannan, and V. K. Prasanna, "Performance modeling and fpga acceleration of homomorphic encrypted convolution," in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*. IEEE, 2021, pp. 115–121.
- [10] Y. Yang, S. R. Kuppannagari, R. Kannan, and V. K. Prasanna, "Fpga accelerator for homomorphic encrypted sparse convolutional neural network inference," in *2022 IEEE 30th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2022, pp. 1–9.
- [11] X. Ren, Z. Chen, Z. Gu, Y. Lu, R. Zhong, W.-J. Lu, J. Zhang, Y. Zhang, H. Wu, X. Zheng *et al.*, "Cham: A customized homomorphic encryption accelerator for fast matrix-vector product," in *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 2023, pp. 1–6.
- [12] Y. Yang, S. R. Kuppannagari, R. Kannan, and V. K. Prasanna, "Bandwidth efficient homomorphic encrypted matrix vector multiplication accelerator on fpga," in *2022 International Conference on Field-Programmable Technology (ICFPT)*. IEEE, 2022, pp. 1–9.
- [13] X. Jiang, M. Kim, K. Lauter, and Y. Song, "Secure outsourced matrix computation and application to neural networks," in *Proceedings of the 2018 ACM SIGSAC conference on computer and communications security*, 2018, pp. 1209–1222.
- [14] Z. Huang, C. Hong, C. Weng, W.-j. Lu, and H. Qu, "More efficient secure matrix multiplication for unbalanced recommender systems," *IEEE Transactions on Dependable and Secure Computing*, vol. 20, no. 1, pp. 551–562, 2021.
- [15] H. Huang and H. Zong, "Secure matrix multiplication based on fully homomorphic encryption," *The Journal of Supercomputing*, vol. 79, no. 5, pp. 5064–5085, 2023.
- [16] Y. Gao, G. Quan, S. Homs, W. Wen, and L. Wang, "Secure and efficient general matrix multiplication on cloud using homomorphic encryption," *The Journal of Supercomputing*, vol. 80, no. 18, pp. 26394–26434, 2024.
- [17] S. Halevi and V. Shoup, "Faster homomorphic linear transformations in helib," in *Annual International Cryptology Conference*. Springer, 2018, pp. 93–120.
- [18] J. H. Cheon, H. Choe, D. Lee, and Y. Son, "Faster linear transformations in HELib, revisited," *IEEE Access*, vol. 7, pp. 50595–50604, 2019.
- [19] K. Han, M. Hhan, and J. H. Cheon, "Improved homomorphic discrete fourier transforms and the bootstrapping," *IEEE Access*, vol. 7, pp. 57361–57370, 2019.
- [20] Z. Xu, Y. Yang, R. Kannan, and V. K. Prasanna, "Bandwidth efficient homomorphic encrypted discrete fourier transform acceleration on fpga," in *2024 IEEE 32nd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2024, pp. 1–12.
- [21] R. Ran, W. Wang, Q. Gang, J. Yin, N. Xu, and W. Wen, "Cryptogen: Fast and scalable homomorphically encrypted graph convolutional network inference," *Advances in Neural information processing systems*, vol. 35, pp. 37676–37689, 2022.
- [22] C. Juvekar, V. Vaikuntanathan, and A. Chandrakasan, "{GAZELLE}: A low latency framework for secure neural network inference," in *27th USENIX security symposium (USENIX security 18)*, 2018, pp. 1651–1669.
- [23] J. Moon, D. Yoo, X. Jiang, and M. Kim, "Thor: Secure transformer inference with homomorphic encryption," *Cryptology ePrint Archive*, 2024.
- [24] Z. Brakerski, "Fully homomorphic encryption without modulus switching from classical gapsvp," in *Annual cryptology conference*. Springer, 2012, pp. 868–886.
- [25] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(leveled) fully homomorphic encryption without bootstrapping," *ACM Transactions on Computation Theory (TOCT)*, vol. 6, no. 3, pp. 1–36, 2014.
- [26] J. Fan and F. Vercauteren, "Somewhat practical fully homomorphic encryption," *Cryptology ePrint Archive*, 2012.
- [27] J. H. Cheon, A. Kim, M. Kim, and Y. Song, "Homomorphic encryption for arithmetic of approximate numbers," in *Advances in cryptology—ASIACRYPT 2017: 23rd international conference on the theory and applications of cryptology and information security, Hong kong, China, December 3-7, 2017, proceedings, part i 23*. Springer, 2017, pp. 409–437.
- [28] J. Kim, G. Lee, S. Kim, G. Sohn, M. Rhu, J. Kim, and J. H. Ahn, "Ark: Fully homomorphic encryption accelerator with runtime data generation and inter-operation key reuse," in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1237–1254.
- [29] J. H. Cheon, K. Han, A. Kim, M. Kim, and Y. Song, "A full rms variant of approximate homomorphic encryption," in *Selected Areas in Cryptography—SAC 2018: 25th International Conference, Calgary, AB, Canada, August 15–17, 2018, Revised Selected Papers 25*. Springer, 2019, pp. 347–368.
- [30] K. Han and D. Ki, "Better bootstrapping for approximate homomorphic encryption," in *Cryptographers' Track at the RSA Conference*. Springer, 2020, pp. 364–390.
- [31] Z. Liang and Y. Zhao, "Number theoretic transform and its applications in lattice-based cryptosystems: A survey," *arXiv preprint arXiv:2211.13546*, 2022.
- [32] W. Jung, S. Kim, J. H. Ahn, J. H. Cheon, and Y. Lee, "Over 100x faster bootstrapping in fully homomorphic encryption through memory-centric optimization with gpus," *IACR Transactions on Cryptographic Hardware and Embedded Systems*, pp. 114–148, 2021.
- [33] S. Kim, J. Kim, M. J. Kim, W. Jung, J. Kim, M. Rhu, and J. H. Ahn, "Bts: An accelerator for bootstrappable fully homomorphic encryption," in *Proceedings of the 49th annual international symposium on computer architecture*, 2022, pp. 711–725.
- [34] AMD, "5TH GEN AMD EPYC™ PROCESSOR ARCHITECTURE," 2025. [Online]. Available: <https://www.amd.com/content/dam/amd/en/documents/epyc-business-docs/white-papers/5th-gen-amd-epyc-processor-architecture-white-paper.pdf>
- [35] J.-P. Bossuat, C. Mouchet, J. Troncoso-Pastoriza, and J.-P. Hubaux, "Efficient bootstrapping for approximate homomorphic encryption with non-sparse keys," in *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. Springer, 2021, pp. 587–617.
- [36] R. Agrawal, L. De Castro, C. Juvekar, A. Chandrakasan, V. Vaikuntanathan, and A. Joshi, "Mad: Memory-aware design techniques for accelerating fully homomorphic encryption," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 685–697.
- [37] S. Kim, K. Lee, W. Cho, J. H. Cheon, and R. A. Rutenbar, "Fpga-based accelerators of fully pipelined modular multipliers for homomorphic encryption," in *2019 International Conference on ReConfigurable Computing and FPGAs (ReConFig)*. IEEE, 2019, pp. 1–8.

- [38] M. S. Riazi, K. Laine, B. Pelton, and W. Dai, "Heax: An architecture for computing on encrypted data," in *Proceedings of the twenty-fifth international conference on architectural support for programming languages and operating systems*, 2020, pp. 1295–1309.
- [39] R. Chen and V. K. Prasanna, "Automatic generation of high throughput energy efficient streaming architectures for arbitrary fixed permutations," in *2015 25th International Conference on Field Programmable Logic and Applications (FPL)*. IEEE, 2015, pp. 1–8.
- [40] A. Ibarrondo and A. Viand, "Pythel: Python for homomorphic encryption libraries," in *Proceedings of the 9th on Workshop on Encrypted Computing & Applied Homomorphic Cryptography*, 2021, pp. 11–16.
- [41] K. Han, S. Hong, J. H. Cheon, and D. Park, "Logistic regression on homomorphic encrypted data at scale," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 33, no. 01, 2019, pp. 9466–9471.
- [42] K. Shivdikar, Y. Bao, R. Agrawal, M. Shen, G. Jonatan, E. Mora, A. Ingare, N. Livesay, J. L. Abellán, J. Kim *et al.*, "Gme: Gpu-based microarchitectural extensions to accelerate homomorphic encryption," in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 670–684.
- [43] A. Al Badawi, L. Hoang, C. F. Mun, K. Laine, and K. M. M. Aung, "Privft: Private and fast text classification with homomorphic encryption," *IEEE Access*, vol. 8, pp. 226 544–226 556, 2020.
- [44] R. Agrawal, L. de Castro, G. Yang, C. Juvekar, R. Yazicigil, A. Chandrakasan, V. Vaikuntanathan, and A. Joshi, "Fab: An fpga-based accelerator for bootstrappable fully homomorphic encryption," in *2023 IEEE International symposium on high-performance computer architecture (HPCA)*. IEEE, 2023, pp. 882–895.
- [45] Y. Yang, H. Zhang, S. Fan, H. Lu, M. Zhang, and X. Li, "Poseidon: Practical homomorphic encryption accelerator," in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 870–881.
- [46] Z. Xu, T. Ye, R. Kannan, and V. K. Prasanna, "Fast: Fpga acceleration of fully homomorphic encryption with efficient bootstrapping," in *Proceedings of the 2025 ACM/SIGDA International Symposium on Field Programmable Gate Arrays*, 2025, pp. 115–126.
- [47] S. Hong, S. Kim, J. Choi, Y. Lee, and J. H. Cheon, "Efficient sorting of homomorphic encrypted data with k-way sorting network," *IEEE Transactions on Information Forensics and Security*, vol. 16, pp. 4389–4404, 2021.
- [48] J.-W. Lee, H. Kang, Y. Lee, W. Choi, J. Eom, M. Deryabin, E. Lee, J. Lee, D. Yoo, Y.-S. Kim *et al.*, "Privacy-preserving machine learning with fully homomorphic encryption for deep neural network," *IEEE Access*, vol. 10, pp. 30 039–30 054, 2022.
- [49] R. Podschwadt and D. Takabi, "Classification of encrypted word embeddings using recurrent neural networks," in *PrivateNLP@ WSDM*, 2020, pp. 27–31.
- [50] D. H. D. K. Mishra and M. Yasuda, "Efficient secure matrix multiplication over lwe-based homomorphic encryption," *Tatra Mt. Math. Publ.*, vol. 67, pp. 69–83, 2016.